

```
tarun@LAPTOP-BRD2JBFC:~/oslab/2520090020_Practical$ ./practical11
```

```
=====
PRACTICAL 11
=====
```

```
Commands:
```

```
race      - Demonstrate race condition
mutex     - Demonstrate mutex synchronization
compare   - Explain synchronization impact
all       - Run race + mutex + comparison
exit      - Exit program
```

```
practical11> all
```

```
=====
RACE CONDITION TEST
=====
```

```
Threads      : 4
Increments/thread: 100000
Expected counter: 400000
```

```
Creating threads using pthread_create()...
All race-condition threads created.
```

```
Waiting using pthread_join()...
Race thread 3 completed.
Race thread 1 completed.
Race thread 2 completed.
Race thread 4 completed.
All threads joined successfully.
```

```
Expected value : 400000
Actual value   : 149497
RESULT: RACE CONDITION DETECTED
Some increments were lost because multiple
threads accessed the shared variable concurrently.
=====
```

```
=====
                        MUTEX TEST
=====
Threads          : 4
Increments/thread: 100000
Expected counter : 400000

Creating synchronized threads...
All mutex threads created.

Waiting using pthread_join()...
Mutex thread 2 completed.
Mutex thread 3 completed.
Mutex thread 1 completed.
Mutex thread 4 completed.
All threads joined successfully.

Expected value : 400000
Actual value   : 400000
RESULT: MUTEX SYNCHRONIZATION SUCCESS
No increments were lost.
=====
```

```

=====
                SYNCHRONIZATION COMPARISON
=====
Race condition:
- Multiple threads update shared counter.
- No mutual exclusion is used.
- Updates can be lost.

Mutex synchronization:
- pthread_mutex_lock() protects the counter.
- Only one thread updates the counter at a time.
- pthread_mutex_unlock() releases the lock.
- Expected value is preserved.

Impact of synchronization:
Mutex improves correctness by preventing
simultaneous access to the critical section.
=====
practical11> race

=====
                RACE CONDITION TEST
=====
Threads          : 4
Increments/thread: 100000
Expected counter: 400000

Creating threads using pthread_create()...
All race-condition threads created.

Waiting using pthread_join()...
Race thread 2 completed.
Race thread 1 completed.
Race thread 3 completed.
Race thread 4 completed.
All threads joined successfully.

Expected value : 400000
Actual value   : 149770
RESULT: RACE CONDITION DETECTED
Some increments were lost because multiple
threads accessed the shared variable concurrently.
=====
practical11> compare

```

```
practical11> compare
```

```
=====
```

SYNCHRONIZATION COMPARISON

```
=====
```

Race condition:

- Multiple threads update shared counter.
- No mutual exclusion is used.
- Updates can be lost.

Mutex synchronization:

- pthread_mutex_lock() protects the counter.
- Only one thread updates the counter at a time.
- pthread_mutex_unlock() releases the lock.
- Expected value is preserved.

Impact of synchronization:

Mutex improves correctness by preventing simultaneous access to the critical section.

```
=====
```

```
practical11>
```